

FenixCMS SaaS - Plan maestro de modificaciones fase por fase

Copia de control del PDF generado en ChatGPT. Documento sin código, pensado para revisar el trabajo fase por fase.

FenixCMS SaaS - Plan maestro de modificaciones fase por fase

FenixCMS - Plan maestro de modificaciones Pagina 1

FenixCMS SaaS

Plan maestro de modificaciones - fase

por fase

Documento de control técnico y operativo, sin código

Repositorio analizado fenixcmssl-spec/moreno

Rama main

Commit de referencia 81b8fb3f0ee73c346adbac6465646faa43659049

Fecha del análisis 30 de septiembre de 2026

Alcance Arquitectura SaaS multi-tenant, PostgreSQL, licencias, planes, pagos, administración, tenant CMS, dominios, idiomas, plugins, temas, media y despliegue.

Este documento parte del contenido real visible en GitHub en la rama main. No reproduce código: describe que hay que cambiar, por que, en que orden, como comprobarlo y que debe considerarse terminado.

1. Como utilizar este documento

La regla de trabajo es una fase cada vez. No se debe saltar a la fase siguiente hasta que el punto de control de la fase actual quede comprobado. La finalidad es que puedas revisar el trabajo realizado en AI Studio, mantener GitHub como referencia y evitar que una pantalla de prueba o un dato en memoria parezca persistente cuando no lo es.

Las etiquetas de estado significan: CRITICO = afecta seguridad, integridad o produccion y debe resolverse antes de considerar el sistema listo; PARCIAL = existe la base pero faltan piezas; EXISTENTE = la capacidad esta presente y debe conservarse mientras se completa la integracion.

Importante: el analisis se ha realizado sobre GitHub main. Hay trabajo de endurecimiento que ya se habia preparado localmente en Debian pero que no estaba incorporado a main en la verificacion anterior. Ese trabajo se recoge en la Fase 1 para no duplicarlo.

2. Fotografia real del repositorio analizado

Area Estado Conclusiones principales

PostgreSQL / Prisma EXISTENTE +
PARCIAL

El esquema usa PostgreSQL y Prisma y cubre usuarios, sesiones, tenants, dominios, aplicaciones, planes, entitlements, licencias, suscripciones, pagos, facturas, productos, pedidos, plugins, temas, media, paginas, blog y clasificados.

Aislamiento
multi-tenant

EXISTENTE +
PARCIAL

Existe una extension de Prisma para imponer tenantId y bloquear cruces.
Debe revisarse que todas las rutas sensibles la utilicen de forma coherente.

Autenticacion / RBAC EXISTENTE +
PARCIAL

Hay sesiones persistentes, roles, guard de SUPER_ADMIN y contexto de tenant. Falta cerrar todos los bordes de la aplicacion y eliminar accesos paralelos locales.

Planes / entitlements CRITICO El esquema persiste PlanEntitlement, pero el seed actual crea dos planes y no siembra registros de entitlement. Esto puede dejar limites efectivos en cero.

Licencias CRITICO Existe validacion PostgreSQL asincrona, pero todavia hay rutas/metodos sincronicos en memoria y el servicio de entitlements conserva un camino de activacion por Tenant.licenseKey.

Provisionamiento
tenant

CRITICO El endpoint actual separa la creacion de licencia, activacion y tenant. La operacion debe ser atomica y recuperable.

Pago SaaS CRITICO El servidor tiene piezas de checkout y verificacion, pero el flujo visible de compra en StoreContext simula PayPal y crea datos localmente.

Persistencia del CMS CRITICO StoreContext inicializa mucha informacion desde INITIAL_* y muchas mutaciones modifican solo estado React. Debe quedar claro que PostgreSQL es la autoridad.

Plugins / temas PARCIAL Hay servicios, APIs y tablas, pero el instalador visible sigue simulando parte de las cargas y usa estado local para algunas operaciones.

Marketplace PARCIAL Existe la interfaz Marketplaceltem y catalogo inicial, pero no existe modelo Prisma persistente para el marketplace.

Branding / favicon PARCIAL La UI tiene logo y favicon y MediaAsset existe, pero updateTenantBranding es local y layout mantiene metadata fija.

Idiomas PARCIAL La base actual contempla ES, IT, EN, FR, DE y PT. El soporte solicitado de Haitian Creole no existe todavia.

Menus / reseñas PARCIAL Existen tipos de menu y ProductReview, pero no aparecen como modelos persistentes equivalentes en Prisma.

El repositorio analizado contiene 191 archivos. Entre sus areas principales se encuentran 67 archivos bajo app, 39 componentes, 42 archivos en lib, 9 elementos de Prisma, 4 scripts y 11 suites de test. No existe README.md en la raiz del repositorio analizado.

FenixCMS - Plan maestro de modificaciones Pagina 3

3. Arquitectura que ya existe y que no se debe romper

- PostgreSQL + Prisma deben ser la fuente de verdad en produccion.
- La extension de aislamiento por tenant debe conservarse; los servicios y rutas deben reforzarla, no

sustituirla por filtros manuales dispersos.

- El guard de SUPER_ADMIN y las sesiones persistentes en PostgreSQL forman la base del acceso al area

SaaS.

- La arquitectura diferencia portal SaaS, login SaaS, storefront del tenant, backoffice del tenant y login del

comprador.

- Los fallbacks en memoria pueden quedar exclusivamente para tests o desarrollo controlado. No pueden

representar una copia silenciosa de produccion.

- El frontend no debe ser la autoridad para licencias, pagos, permisos, limites, provisioning ni persistencia

de datos.

4. Hallazgos que deben estar visibles durante todo el proyecto

- Un tenant activo con un texto licenseKey no debe considerarse licenciado si no existe una License valida y activa.

- Un plan sin PlanEntitlement no puede interpretarse accidentalmente como un plan ilimitado o como un permiso implicito.

- Una compra de licencia no puede ser un cambio de estado React: debe ser un flujo de servidor verificable, persistente y auditable.

- Un CRUD no esta terminado cuando cambia la pantalla; esta terminado cuando crea/lee/actualiza/elimina el registro correcto en PostgreSQL y puede comprobarse desde una segunda sesion o desde otro proceso.

- Un paquete de plugin o tema no esta instalado por leer el nombre del fichero: debe ser validado, persistido, asociado al tenant y gobernado por permisos y entitlements.

- La metadata de la pagina no puede permanecer fija cuando el storefront es multi-tenant.

FenixCMS - Plan maestro de modificaciones Pagina 4

5. Plan completo por fases

El plan siguiente esta ordenado por dependencias. Las primeras fases corrigen integridad y gobierno de datos; despues se consolidan negocio SaaS, administracion y tenant; al final se hace aceptacion y despliegue.

FASE 0 - Auditoria de referencia y congelacion del alcance

Ejecucion prevista: ORDENADOR DEBIAN + GITHUB

Estado detectado en GitHub main: EXISTENTE

Objetivo: Fijar una fotografia unica del estado real antes de modificar nada.

Situacion actual: El repositorio main actual es 81b8fb3... y el analisis se ha hecho sobre esa rama. Existe una auditoria previa de AI Studio, pero su atribucion exacta de algunos commits no coincide con el historial real de GitHub; por eso GitHub debe mandar para el estado de codigo.

Modificaciones que deben realizarse:

- Guardar el commit de referencia y la fecha del analisis como baseline del proyecto.
- Mantener separadas tres referencias: codigo de GitHub, trabajo pendiente/local de Debian y despliegue

real del VPS.

- No usar la pantalla Preview de AI Studio como prueba de persistencia de negocio.
- Registrar que los cambios ya hechos localmente deben integrarse de forma controlada antes de seguir

con nuevas modificaciones.

Punto de control para el usuario:

- Puedes identificar en cualquier momento cual es el commit de referencia.
- No existe una segunda fuente no documentada de cambios.
- Cada fase posterior puede señalar claramente que archivos o subsistemas toca.

Criterio de cierre:

- Baseline documentado, sin cambios funcionales introducidos en esta fase.
- Lista de riesgos critica visible antes de tocar el proyecto.

FASE 1 - Consolidacion del trabajo AI Studio / Debian / GitHub

Ejecucion prevista: ORDENADOR DEBIAN + GITHUB

Estado detectado en GitHub main: CRITICO

Objetivo: Evitar duplicar correcciones ya preparadas y llevar a una sola linea de cambios las mejoras aprobadas.

Situacion actual: Se habia preparado localmente una rama de endurecimiento que incluia: eliminacion del fail-open de Tenant.licenseKey, rethrow de errores de DB en produccion, provisioning transaccional, metodos async de licencia, proteccion de planes y test de regresion. Esos cambios no estaban en main en la ultima verificacion.

Modificaciones que deben realizarse:

- Comparar el trabajo local con origin/main y separar cambios terminados, incompletos y obsoletos.
- Revisar uno por uno los cambios locales de licencia, entitlements, pagos, webhook, provisioning, planes,

storefront y tests.

- Integrar primero las correcciones de integridad que ya existen y que sigan siendo validas.
- Evitar que AI Studio vuelva a implementar un problema que ya fue resuelto en Debian.
- Crear una convencion de trabajo: una fase, una revision, una validacion y despues commit.

FenixCMS - Plan maestro de modificaciones Pagina 5

Dependencias y orden:

Esta fase precede a las fases 2-9 porque esas fases dependen de saber que endurecimientos ya estan incorporados.

Punto de control para el usuario:

- GitHub main y la rama de trabajo deben quedar reconciliados sin perder cambios validos.
- La correccion de licensing fail-open debe quedar presente en la rama objetivo.
- El provisioning transaccional preparado previamente debe quedar identificado como integrado o

pendiente, nunca duplicado.

Criterio de cierre:

- Un solo conjunto de cambios objetivo.
- No hay correcciones criticas conocidas esperando ser copiadas a mano otra vez.

FASE 2 - PostgreSQL como fuente unica de verdad en produccion

Ejecucion prevista: AI STUDIO para codigo + ORDENADOR DEBIAN para control

Estado detectado en GitHub main: CRITICO

Objetivo: Hacer imposible que el sistema de produccion vuelva a una copia de memoria cuando PostgreSQL falla o esta vacio.

Situacion actual: lib/prisma.ts ya distingue produccion de desarrollo/test y contiene un InMemoryTestPrisma. El objetivo es conservar ese soporte solo para pruebas y evitar que servicios de negocio utilicen INITIAL_* como autoridad de produccion.

Modificaciones que deben realizarse:

- Revisar todos los servicios y rutas que todavia importan INITIAL_APPLICATIONS, INITIAL_PLANS,

INITIAL_TENANTS, INITIAL_LICENSES u otros INITIAL_* para decidir si su uso es solo test/dev o si debe desaparecer del camino de produccion.

- Un error de conexion, timeout, fallo de lectura o inconsistencia de PostgreSQL debe producir error controlado y visible, no datos demo.
- Mantener los fallbacks de test identificados como tales y aislados de produccion.
- Revisar caches para que no se conviertan en una segunda base de datos: deben tener TTL e invalidacion

y nunca reconstruir una realidad comercial distinta.

Punto de control para el usuario:

- Con PostgreSQL disponible, las respuestas vienen de DB.
- Con DB indisponible en produccion, la peticion falla de forma explicita y auditable.
- No aparece ninguna tienda, plan, licencia o pago ficticio por silencio.

Criterio de cierre:

- No existe fail-open de persistencia en produccion.
- El soporte en memoria esta claramente limitado a test/dev.

FASE 3 - Aislamiento multi-tenant, autenticacion y RBAC

Ejecucion prevista: AI STUDIO para codigo + ORDENADOR DEBIAN para revision

Estado detectado en GitHub main: EXISTENTE + PARCIAL

Objetivo: Garantizar que cada tenant solo pueda operar sobre sus propios datos y que SUPER_ADMIN sea la unica identidad con bypass global.

FenixCMS - Plan maestro de modificaciones Pagina 6

Situacion actual: La extension de Prisma ya aplica tenantId y bloquea cruces; existen SessionService, TenantContext y admin-guard. Debe cerrarse cualquier ruta que siga aceptando tenantId externo sin comprobar contexto, y cualquier acceso que use estados locales del navegador.

Modificaciones que deben realizarse:

- Auditar todas las APIs por categoria: SaaS global, tenant admin y storefront publico.
- Asegurar que las rutas globales aceptan bypass solo para SUPER_ADMIN o procesos de sistema

explicitamente definidos.

- Asegurar que las rutas tenant toman el tenant del contexto autenticado, del dominio verificado o de una

relacion de membership validada.

- Confirmar que cambiar de tenant solo es posible mediante una membresia activa o por SUPER_ADMIN.
- Revisar que un usuario suspendido o una sesion revocada no puedan continuar operando.
- Eliminar rutas paralelas que permitan saltarse el servidor usando licencias almacenadas en memoria del

cliente.

Punto de control para el usuario:

- Tenant A nunca puede leer, editar o borrar registros de Tenant B.
- SUPER_ADMIN puede operar de forma global solo a traves de rutas que lo permitan explicitamente.
- Un usuario tenant no puede escalar a SUPER_ADMIN cambiando datos del navegador.

Criterio de cierre:

- Pruebas cruzadas de tenant pasan.
- Todas las APIs sensibles tienen una politica de acceso explicita.

FASE 4 - Catalogo de aplicaciones y modulos

Ejecucion prevista: AI STUDIO

Estado detectado en GitHub main: EXISTENTE + PARCIAL

Objetivo: Hacer que E-commerce, Blog, Blog+Ads, Classifieds y futuras aplicaciones sean datos reales del catalogo SaaS y no claves sueltas repartidas por el frontend.

Situacion actual: El seed crea ECOMMERCE, BLOG, BLOG_ADS y CLASSIFIEDS. Types contempla mas claves, incluyendo BOOKING, LMS, DIRECTORY y otras, pero no todas existen como aplicaciones sembradas. ApplicationService y su API ya permiten gestionarlasy.

Modificaciones que deben realizarse:

- Definir el catalogo de aplicaciones que entrara en la primera version comercial.
- Mantener Application y ApplicationModule como autoridad para saber que software existe y que modulos

ofrece.

- Asegurar que cada plan pertenece a una aplicacion valida.
- Evitar que un plan de una aplicacion se pueda asignar a un tenant con otra aplicacion.
- Preparar modulos para que una aplicacion futura pueda compartir el motor de tenant, licencia,

entitlements, media, usuarios y temas.

Punto de control para el usuario:

- Cada aplicacion visible en Super Admin existe en PostgreSQL.
- Cada plan muestra la aplicacion a la que pertenece.
- No hay ids ficticios de aplicacion usados como autoridad comercial.

Criterio de cierre:

FenixCMS - Plan maestro de modificaciones Pagina 7

- Catalogo de aplicaciones definido y persistente.
- Compatibilidad aplicacion-plan validada.

FASE 5 - Planes, precios y matriz de entitlements

Ejecucion prevista: AI STUDIO

Estado detectado en GitHub main: CRITICO

Objetivo: Convertir los planes en una matriz comercial real y persistente, con limites y funciones que el motor pueda aplicar de forma determinista.

Situacion actual: El schema tiene PlanEntitlement, pero el seed actual crea plan_starter y plan_growth sin sembrar entitlements. En el VPS auditado anteriormente los planes aparecian con cero entitlements, provocando limites efectivos a cero.

Modificaciones que deben realizarse:

- Definir el conjunto de planes comerciales definitivo. El código actual contempla conceptos Starter,

Pro/Growth y Enterprise; el nombre y precio finales deben ser una decisión de producto, no del sistema.

- Definir para cada plan los límites numéricos: productos, almacenamiento, dominios, usuarios y cualquier

recurso adicional.

- Definir funciones booleanas: IA, plugins, custom domain, tema personalizado, blog, anuncios y otras

capacidades que realmente vayan a existir.

- Definir listas como `plugins.allowed` cuando el negocio necesite restricciones por plugin.

- Sembrar los entitlements en PostgreSQL y hacer que el seed sea idempotente.

- Evitar que una ausencia de entitlement se interprete como permiso; el motor debe mantener

deny-by-default.

- Asegurar que el panel de Entitlements crea, edita y elimina registros reales.

Punto de control para el usuario:

- Un plan recién creado tiene una ficha de entitlements comprobable en DB.

- Cambiar un límite en el panel cambia el comportamiento de un tenant tras la siguiente lectura.

- Un tenant sin licencia válida no obtiene capacidades por tener `planId`.

Criterio de cierre:

- No hay planes comerciales con entitlements accidentalmente vacíos.

- La matriz de capacidades es comprensible y verificable por un administrador.

FASE 6 - Motor de ciclo de vida de licencias

Ejecución prevista: AI STUDIO

Estado detectado en GitHub main: CRÍTICO

Objetivo: Hacer de License la única autoridad de acceso comercial junto con su suscripción cuando corresponda.

Situación actual: LicenseService ya dispone de hash SHA-256, claves de alta entropía y validación async contra PostgreSQL. Todavía existe código sincrónico en memoria y, sobre GitHub main, EntitlementService contiene el camino que activa licencia por `Tenant.licenseKey`.

Modificaciones que deben realizarse:

- Eliminar del camino de producción toda decisión de validez basada exclusivamente en `Tenant.licenseKey`.

- Conservar `Tenant.licenseKey` solo como referencia si el modelo lo necesita, nunca como prueba de

licencia.

FenixCMS - Plan maestro de modificaciones Página 8

- Usar validación async PostgreSQL para `validate`, `activate`, `deactivate`, `renew` y cambios de estado utilizados por rutas de producción.

- Comprobar estado, expiración, tenant, aplicación, plan y activaciones antes de conceder acceso.

- Registrar las transiciones importantes: creada, activada, suspendida, renovada, expirada, revocada y

cancelada.

- Asegurar que la cuota de activaciones se comprueba atómicamente para evitar carreras.

- Crear regresión explícita para el caso tenant activo + `licenseKey` de texto + cero License rows. El

resultado debe ser acceso denegado.

Punto de control para el usuario:

- Una License válida existe en DB y su `tenantId` coincide.

- El límite de dominios no se puede saltar.

- Una licencia expirada o revocada deja de conceder acceso inmediatamente.

Criterio de cierre:

- No existe ninguna ruta de produccion que trate Tenant.licenseKey como licencia valida por si sola.
- Las decisiones de entitlements parten de una licencia o suscripcion efectiva real.

FASE 7 - Provisionamiento atomico de tenant

Ejecucion prevista: AI STUDIO

Estado detectado en GitHub main: CRITICO

Objetivo: Hacer que la compra o alta de un comercio sea todo-o-nada.

Situacion actual: El endpoint actual crea license, activation y tenant en pasos separados. Ya se habia preparado localmente una funcion transaccional TenantService.provisionTenantWithLicenseAsync() que debe integrarse si sigue siendo valida.

Modificaciones que deben realizarse:

- Un unico flujo debe coordinar Tenant, License, LicenseActivation, Domain, User propietario y

TenantMembership cuando formen parte del alta.

- Comprobar primero Application y Plan y su compatibilidad.
- Generar la clave de licencia de forma criptograficamente segura y almacenar el hash para validacion.
- Crear el tenant y sus relaciones dentro de una unica transaccion cuando sea posible.
- Si una parte falla, ninguna parte del alta comercial debe quedar parcialmente creada.
- Hacer el proceso idempotente para evitar duplicados por reintentos de pago o refresh del navegador.
- Definir claramente el dominio inicial y su relacion con LicenseActivation.

Punto de control para el usuario:

- Un fallo en cualquier paso deja el sistema sin un tenant incompleto.
- Repetir el mismo evento no crea un segundo comercio accidental.
- El tenant creado tiene license, plan, aplicacion y dominio coherentes.

Criterio de cierre:

- Alta transaccional validada con rollback.
- Idempotencia demostrada.

FASE 8 - Checkout y compra de licencia desde el portal SaaS

Ejecucion prevista: AI STUDIO

FenixCMS - Plan maestro de modificaciones Pagina 9

Estado detectado en GitHub main: CRITICO

Objetivo: Sustituir la simulacion de compra del navegador por un flujo de servidor real.

Situacion actual: SaasLanding llama a StoreContext.buyLicenseWithPayPal. En main, ese metodo genera una clave con Math.random, crea un tenant en memoria y simula una espera; no pasa por el flujo de pago/verification/provisioning de PostgreSQL.

Modificaciones que deben realizarse:

- Eliminar la simulacion de compra como autoridad comercial.
- El boton de compra debe iniciar un checkout server-side usando el plan real de PostgreSQL.
- El frontend no debe generar claves de licencia ni decidir precios finales.
- El estado visible de pago debe depender de una confirmacion verificable del proveedor.
- Al completar pago, el servidor debe crear o renovar suscripcion y licencia mediante el flujo atomico

definido en fases anteriores.

- El cliente debe recibir solo el resultado necesario para continuar, no secretos internos ni datos que permitan falsificar estado.

Punto de control para el usuario:

- Un mismo checkout visto desde dos navegadores refleja el mismo estado.

- Modificar el precio en el navegador no altera el importe autorizado.
- No se crea una licencia si el proveedor no confirma el pago.

Criterio de cierre:

- Compra real y persistente.
- No quedan caminos de compra solo-local en produccion.

FASE 9 - Webhooks, pagos, suscripciones y facturación

Ejecución prevista: AI STUDIO + VPS para pruebas reales

Estado detectado en GitHub main: CRITICO

Objetivo: Cerrar el circuito económico de forma idempotente y auditable.

Situación actual: WebhookService ya verifica firmas de Stripe y tiene validación de PayPal; existen Payment, Subscription, Invoice y WebhookEvent en Prisma. Persisten fallbacks y flujos separados que deben quedar estrictamente como test/dev.

Modificaciones que deben realizarse:

- Verificar que el evento del proveedor se autentica antes de cambiar estados comerciales.
- Registrar WebhookEvent y hacer idempotente el procesamiento por eventId.
- Asegurar que un mismo evento no crea doble suscripción, doble licencia ni doble factura.
- Definir el orden de estado: pago confirmado, suscripción efectiva, licencia efectiva, factura emitida y

audit log.

- Resolver los estados de pago fallido, reembolsado, suscripción cancelada y pago atrasado.
- Eliminar el uso silencioso de ledgers de memoria para decisiones de producción.
- Documentar claramente que proveedores están realmente habilitados y cuáles solo tienen scaffolding.

Punto de control para el usuario:

- Duplicar un webhook no duplica recursos.
- El estado de License sigue el de Subscription cuando corresponda.
- Una factura persistente permite rastrear pago, tenant y suscripción.

FenixCMS - Plan maestro de modificaciones Pagina 10

Criterio de cierre:

- Flujo de pago completo e idempotente.
- Eventos de proveedor trazables por AuditLog y WebhookEvent.

FASE 10 - Consola Super Admin

Ejecución prevista: AI STUDIO

Estado detectado en GitHub main: PARCIAL

Objetivo: Hacer que la consola SaaS sea un panel operativo real y no una combinación de datos de demostración y operaciones locales.

Situación actual: SuperAdminPortal tiene tabs para aplicaciones, planes, entitlements, licencias, activaciones, suscripciones, pagos, facturas, tenants, dominios, temas, plugins, usuarios, auditoría y ajustes. Algunas cifras iniciales son demo y la creación de licencias usa StoreContext local.

Modificaciones que deben realizarse:

- Eliminar cifras demo como fuente visible cuando el API no las ha confirmado.
- Hacer que crear, editar, suspender, renovar y revocar licencias use las APIs protegidas.
- Hacer que EntitlementsManager persista realmente los cambios.
- Asegurar protección SUPER_ADMIN para todas las operaciones globales, incluidos los endpoints de

planes, entitlements, licencias y catálogos.

- Asegurar que los métodos de servicio también validan reglas críticas, por ejemplo que PlanService.delete

comprueba canDelete internamente.

- Mostrar errores de DB de forma explicita, sin reemplazarlos por listas demo.

Punto de control para el usuario:

- Una accion hecha en Super Admin sobrevive a recargar y a otra sesion.
- Los endpoints rechazan peticiones sin SUPER_ADMIN.
- Las cifras del dashboard representan DB y no defaults fijos.

Criterio de cierre:

- Super Admin completamente persistente y protegido.

FASE 11 - Eliminacion de la falsa persistencia del StoreContext

Ejecucion prevista: AI STUDIO

Estado detectado en GitHub main: CRITICO

Objetivo: Dejar de usar el estado React como base de datos del CMS.

Situacion actual: StoreContext tiene 978 lineas y crea arrays iniciales para planes, licencias, pedidos, media, plugins, temas y otros recursos. Muchas funciones de mutacion solo hacen setState y AuditLog local.

Modificaciones que deben realizarse:

- Definir el servidor como autoridad para cada entidad comercial.
- Reemplazar mutaciones locales por llamadas a sus APIs existentes o crear API solo donde realmente falta.
- Despues de una mutacion, volver a leer el recurso o usar la respuesta persistida del servidor para

actualizar el estado.

- Evitar que un fallo de API haga creer al usuario que el cambio se guardo.
- Revisar especialmente updatePlan, createPlan, deletePlan, entitlements, licencias, tenant settings,

productos, blog, media, pedidos, plugins y temas.

FenixCMS - Plan maestro de modificaciones Pagina 11

- Mantener solo en memoria lo que sea estrictamente interfaz temporal, por ejemplo abrir un modal o el carrito del usuario anonimo cuando no deba persistirse.

Punto de control para el usuario:

- Abrir una segunda sesion muestra el mismo dato.
- Cerrar y volver a entrar no borra un CRUD correctamente guardado.
- Los errores de backend no dejan UI en estado de exito falso.

Criterio de cierre:

- StoreContext orquesta estado de UI; PostgreSQL y sus APIs son la fuente de verdad.

FASE 12 - CRUD completo del tenant CMS

Ejecucion prevista: AI STUDIO

Estado detectado en GitHub main: CRITICO

Objetivo: Conectar de forma coherente productos, categorias, pedidos, clientes, cupones, paginas, blog y clasificados con PostgreSQL.

Situacion actual: Existen muchas APIs de tenant ya creadas. Sin embargo, StoreContext aun contiene operaciones locales para varios recursos. Debe hacerse una auditoria entidad por entidad para que todas tengan el mismo comportamiento de persistencia.

Modificaciones que deben realizarse:

- Inventariar cada recurso y su API existente.
- Confirmar tenantId, autenticacion, permisos y entitlement en todas las rutas de escritura.
- Aplicar la misma politica a crear, editar, eliminar, listar y operaciones bulk.
- Controlar slug unico por tenant y referencias a categoria/cliente/producto.

- Asegurar transacciones donde una accion modifique varias tablas.
- Revisar importaciones masivas para que no creen recursos en el tenant equivocado.

Punto de control para el usuario:

- Cada CRUD puede verificarse en PostgreSQL.
- No hay operaciones locales ocultas que creen datos solo para el navegador.

Criterio de cierre:

- Matriz CRUD completa por entidad y tenant.

FASE 13 - Mediateca y almacenamiento persistente

Ejecucion prevista: AI STUDIO + VPS

Estado detectado en GitHub main: PARCIAL

Objetivo: Convertir la carga de archivos en un flujo seguro, persistente y multi-tenant.

Situacion actual: MediaAsset y StorageService ya existen, con validacion de MIME/tamano, storageKey por tenant y endpoints GET/POST/PATCH/DELETE. MediaLibraryModal, no obstante, contiene archivos seed y actualizaciones optimistas/locales.

Modificaciones que deben realizarse:

- Mantener MediaAsset como registro persistente y StorageService como unica capa de almacenamiento.
- Definir el almacenamiento de produccion: filesystem persistente, volumen, objeto compatible S3 u otro

proveedor, y su politica de backup.

- Eliminar placeholders demo como fuente automatica en produccion.

FenixCMS - Plan maestro de modificaciones Pagina 12

- Validar tipo, tamano, nombre, metadatos y tenant antes de guardar.
- Evitar que una URL externa parezca un archivo local persistido si el sistema no lo ha almacenado.
- Asegurar borrado coordinado entre registro DB y objeto fisico cuando corresponda.

Punto de control para el usuario:

- Subir un archivo y reiniciar el servicio no lo hace desaparecer.
- Un tenant no puede ver media de otro tenant.
- Logo y favicon seleccionados apuntan a assets persistentes.

Criterio de cierre:

- Media real, persistente, aislada y recuperable.

FASE 14 - Branding dinamico: logo, logo movil y favicon

Ejecucion prevista: AI STUDIO

Estado detectado en GitHub main: PARCIAL

Objetivo: Cumplir de verdad el requisito de identidad visual por tenant.

Situacion actual: TenantBrandingSettings ofrece logo y favicon, pero updateTenantBranding en StoreContext es local. app/layout.tsx tiene metadata fija y lang=en. No hay metadata dinamica de favicon/locale por tenant.

Modificaciones que deben realizarse:

- Persistir branding por tenant a traves de la API y PostgreSQL.
- Definir campos de logo principal, logo movil y favicon de forma coherente.
- Conectar selector de Media Library con el asset persistente real.
- Aplicar el branding al storefront y al backoffice del tenant donde corresponda.
- Generar metadata/favicons dinamicos segun el tenant resuelto.
- Eliminar URLs de placeholder de Unsplash en produccion como defaults visibles.
- Asegurar que el cambio de branding invalida el cache de storefront.

Punto de control para el usuario:

- Cada tenant puede tener su logo y favicon independientes.
- Un cambio queda visible tras reinicio.
- El favicon servido corresponde al dominio/tenant actual.

Criterio de cierre:

- Branding persistente y dinamico por tenant.

FASE 15 - Motor de temas y constructor visual

Ejecucion prevista: AI STUDIO

Estado detectado en GitHub main: PARCIAL

Objetivo: Completar la gestion de temas para que crear, subir, editar, previsualizar y activar sea persistente y seguro.

Situacion actual: ThemeService ya gestiona Theme, ThemeInstallation, activacion, draft y publish. MerchantAdmin aun contiene un flujo de upload que toma el nombre del fichero y genera un tema local en memoria.

Modificaciones que deben realizarse:

FenixCMS - Plan maestro de modificaciones Pagina 13

- Usar ThemeService/API como autoridad para crear y editar temas.
- Definir un formato de paquete de tema y una validacion de manifest/compatibilidad.
- Registrar version, autor, aplicacion compatible, secciones y recursos del tema.
- Separar borrador de version publicada.
- Instalar un tema en un tenant mediante ThemeInstallation persistente.
- Evitar que un tenant pueda activar un tema no compatible o no disponible.
- Asegurar rollback a un tema estable si una activacion falla.

Punto de control para el usuario:

- Crear tema desde un navegador y verlo desde otra sesion.
- Activar un tema cambia el storefront despues de recargar.
- Un borrador no se publica hasta una accion explicita.

Criterio de cierre:

- Tema completo: persistencia + instalacion + activacion + draft/publish.

FASE 16 - Motor de plugins

Ejecucion prevista: AI STUDIO

Estado detectado en GitHub main: PARCIAL

Objetivo: Convertir la idea de plugins tipo WordPress en un mecanismo real de instalacion, configuracion y control por plan.

Situacion actual: PluginService persiste Plugin y PluginInstallation y valida manifest. La tabla Plugin incluye tenantId, lo que significa que el actual modelo es mas parecido a un catalogo ya instalado por tenant que a un catalogo global de extensiones.

Modificaciones que deben realizarse:

- Decidir la separacion conceptual entre catalogo global de plugins y instalacion por tenant.
- Si el catalogo global forma parte del producto, crear la entidad persistente adecuada o definir un modelo

equivalente, sin mezclarlo con PluginInstallation.

- Validar manifest, version, compatibilidad de aplicacion, dependencias, permisos y hooks antes de

instalar.

- Controlar que el plan permita el plugin antes de instalar o activar.
- Persistir configuracion y estado de cada instalacion.

- Implementar carga real de paquetes si se permite subir ZIP, con inspeccion y restricciones de seguridad.
- Registrar instalacion, activacion, desactivacion, actualizacion y eliminacion en AuditLog.

Punto de control para el usuario:

- Instalar un plugin no modifica otros tenants.
- Un plan que no permite un plugin no puede activarlo aunque el archivo sea valido.
- La configuracion persiste y se recupera del servidor.

Criterio de cierre:

- Plugin engine gobernado por manifest + tenant + entitlement + persistencia.

FASE 17 - Marketplace persistente de plugins y temas

Ejecucion prevista: AI STUDIO

Estado detectado en GitHub main: PARCIAL

FenixCMS - Plan maestro de modificaciones Pagina 14

Objetivo: Dar al portal comercial un catalogo real de addons si el marketplace va a ser parte del producto final.

Situacion actual: MarketPlaceItem existe en types y INITIAL_MARKETPLACE_ITEMS, pero no existe un modelo Prisma equivalente. SaaS Landing muestra catalogo y SuperAdminPortal permite editar elementos localmente.

Modificaciones que deben realizarse:

- Definir si el marketplace entra en la primera version comercial o se separa como release posterior.
- Si entra, persistir productos de marketplace, versiones, precio, tipo, autor, compatibilidad, estado de

publicacion y archivo/asset.

- Relacionar cada venta con tenant, payment, producto comercial y entrega de instalacion.
- Asegurar que comprar un addon no concede permisos fuera de los entitlements adquiridos.
- Registrar reembolsos, renovaciones y versiones cuando haya suscripcion o actualizacion.

Punto de control para el usuario:

- El catalogo sobrevive a reinicio.
- Comprar un addon deja una traza persistente.
- Super Admin y tenant ven estados coherentes.

Criterio de cierre:

- Marketplace DB-backed o decision documentada de posponerlo.

FASE 18 - Dominios, subdominios, custom domains y SSL

Ejecucion prevista: AI STUDIO + VPS

Estado detectado en GitHub main: PARCIAL

Objetivo: Asegurar que el dominio sea una frontera de tenant fiable y operativa.

Situacion actual: DomainService ya resuelve hostname por DB y createDomain aplica entitlements. middleware clasifica SaaS root, subdominio y custom domain. Falta cerrar el proceso de verificacion real, SSL y sincronizacion de estados.

Modificaciones que deben realizarse:

- Definir el ciclo completo: dominio solicitado, DNS verificado, dominio activo, SSL pendiente/activo/error.
- Separar claramente subdominio de plataforma y dominio personalizado.
- No activar un custom domain sin una prueba de control/propiedad definida.
- Mantener un solo dominio primario por tenant.
- Hacer que el resolver de storefront use el dominio verificado y activo como prioridad.
- Invalidar cache cuando cambia el dominio primario.

Punto de control para el usuario:

- El mismo hostname siempre resuelve al tenant correcto.
- Un dominio de otro tenant no puede adoptarse por parametros manuales.
- El estado SSL mostrado coincide con el estado real de despliegue cuando se integre con el proveedor.

Criterio de cierre:

- Pipeline de dominio documentado y comprobable.

FASE 19 - Sistema multidioma e incorporacion de Haitian Creole

Ejecucion prevista: AI STUDIO

FenixCMS - Plan maestro de modificaciones Pagina 15

Estado detectado en GitHub main: PARCIAL

Objetivo: Completar el requisito de idiomas y evitar que el soporte sea solo un selector visual.

Situacion actual: El repositorio soporta es, it, en, fr, de y pt. types e i18n lo reflejan. El formulario de carga de idiomas existe, pero registrar/subir un idioma no persiste una nueva configuracion real. Haitian Creole (ht) no esta soportado.

Modificaciones que deben realizarse:

- Incorporar el locale ht en tipos, registro de idiomas, diccionario y mecanismos de fallback.
- Definir el numero maximo de idiomas permitido por plan si el negocio asi lo decide.
- Persistir en tenant defaultLocale y supportedLocales.
- Crear una forma real de administrar traducciones y comprobar claves faltantes.
- Evitar que una traduccion ausente rompa la pagina: fallback claro al idioma por defecto.
- Asegurar que storefront, backoffice y emails/plantillas que entren en alcance respeten locale.

Punto de control para el usuario:

- ht aparece como idioma seleccionable y persistente.
- Cambiar idioma sobrevive a logout/reinicio.
- No quedan pantallas criticas con texto inaccesible por falta de traduccion.

Criterio de cierre:

- Sistema de idiomas con ES/IT/EN/FR/DE/PT/HT y cobertura controlada.

FASE 20 - Resolucion del storefront y coherencia de licencia

Ejecucion prevista: AI STUDIO

Estado detectado en GitHub main: CRITICO

Objetivo: Asegurar que hostname -> tenant -> tema -> branding -> contenido -> licencia sea una cadena coherente y DB-first.

Situacion actual: StorefrontService hace resolucion por PostgreSQL y filtra productos, categorias, paginas, blog y clasificados por tenantId. Sin embargo, la validacion de licencia al final aun usa metodos sincronicos de LicenseService en memoria.

Modificaciones que deben realizarse:

- Pasar la validacion final de licencia a la version async PostgreSQL.
- Asegurar que la respuesta del storefront no se construye con datos demo en produccion.
- Mantener el cache como optimizacion y no como autoridad.
- Asegurar que branding, locale, tema, aplicaciones y entitlements usados por storefront vienen del tenant

correcto.

- Denegar o degradar de forma definida un storefront cuya licencia no sea efectiva.

Punto de control para el usuario:

- Cambiar hostname cambia tenant solo cuando el dominio es valido.
- Un tenant suspendido no obtiene storefront plenamente operativo si la politica comercial lo impide.
- El storefront no usa licenseKey textual como prueba suficiente.

Criterio de cierre:

- Resolucion y acceso comercial totalmente DB-first.

FASE 21 - Auditoria de completitud del CMS: paginas, menus, reseñas y contenidos

FenixCMS - Plan maestro de modificaciones Pagina 16

Ejecucion prevista: AI STUDIO

Estado detectado en GitHub main: PARCIAL

Objetivo: Cerrar las diferencias entre lo que types y UI prometen y lo que Prisma realmente persiste.

Situacion actual: types/index.ts contiene CMSMenu y ProductReview, pero el schema Prisma visible no contiene modelos equivalentes. Tambien existen recursos de blog y clasificados. Hay que decidir que entra en el producto base y que se pospone.

Modificaciones que deben realizarse:

- Comparar todas las interfaces de types con modelos Prisma y APIs reales.
- Crear persistencia para menus si el CMS promete menus editables.
- Crear persistencia para reseñas si el e-commerce va a mostrarlas y administrarlasy.
- Revisar homepage, paginas, blog, categorias, SEO y relaciones de contenido.
- Eliminar del roadmap las funcionalidades que solo sean mock visual o marcarlas como futuras.

Punto de control para el usuario:

- Cada funcion anunciada como disponible tiene DB + API + UI coherentes.
- No se presenta una capacidad como persistente si solo existe en types o memoria.

Criterio de cierre:

- Matriz funcional del CMS cerrada y sin promesas fantasma.

FASE 22 - SEO, sitemap y metadata por tenant

Ejecucion prevista: AI STUDIO

Estado detectado en GitHub main: EXISTENTE + PARCIAL

Objetivo: Hacer que SEO sea dinamico, persistente y coherente con el dominio y contenido de cada tenant.

Situacion actual: Existe SEO automation, generacion AI y sitemap. BlogPost y Page tienen campos SEO. Root layout mantiene metadata fija y middleware excluye sitemap/robots de forma global.

Modificaciones que deben realizarse:

- Completar title, description, canonical, Open Graph, Twitter y JSON-LD segun tenant y pagina.
- Asegurar sitemap por hostname/tenant y no mezclar contenido de tenants.
- Relacionar branding y sitio con metadata dinamica.
- Revisar que robots y canonical respeten custom domains.
- Mantener la generacion AI como asistente, nunca como sustituto de la persistencia y validacion manual.

Punto de control para el usuario:

- El sitemap de un tenant no lista URLs de otro.
- Canonical coincide con el dominio que sirve el contenido.
- Favicon, title y social metadata corresponden al tenant.

Criterio de cierre:

- SEO multi-tenant comprobado.

FASE 23 - Auditoria, logs y observabilidad operativa

Ejecucion prevista: AI STUDIO + VPS

Estado detectado en GitHub main: PARCIAL

Objetivo: Poder explicar que ocurrio ante un pago, alta, cambio de plan, acceso, error de DB o instalacion.

Situacion actual: AuditLog y WebhookEvent existen. Falta asegurar que toda operacion critica pasa por ellos y que los errores de DB no se esconden detras de fallback local.